

Take-Home Exercise - Tech Lead Hiring

MarketScope: Portfolio Universe & Store Discovery

1. Context

You're building the first slice of **MarketScope**, a tool that helps a retail/CPG business understand where its own stores ("portfolio") sit relative to the broader universe of retail outlets in a city, and which of those outlets are worth targeting for expansion or route planning.

This exercise focuses on one self-contained workflow: uploading a portfolio, defining a market boundary, discovering stores inside it, and visualizing everything on a dashboard.

2. What We're Evaluating

This is not primarily a "does it run" test. We're looking at:

- **Architecture** - how you structure the backend (layering, separation of concerns, data model), and whether the design would hold up if this became a real product.
- **Code quality** - readability, naming, error handling, consistency.
- **API/integration handling** - how you deal with a real third-party API: rate limits, pagination, partial failures, retries.
- **Data modeling** - how you represent markets, boundaries, stores, and their relationships in the DB.
- **Tests** - unit tests are not mandatory but are a strong positive signal, especially around boundary filtering, header validation, and any geo logic.
- **UX judgment** - the map/dashboard doesn't need to be polished, but it should be usable and the filtering should make sense.

You're free to use AI tools while building this. The bar is on the output: code you can explain and defend in a follow-up conversation, not just code that compiles.

Time box: 2–3 days of effort. We'd rather see a smaller scope done well than everything done shallowly, see Section 6 for what's in vs. out of scope.

3. Tech Stack

- **Frontend:** React preferred (any modern framework is acceptable).
- **Backend:** Python or Node.js preferred (any modern framework is acceptable).
- **Database:** PostgreSQL preferred (any SQL database is acceptable).
- **Maps:** Google Maps JS API or OpenStreetMap/Leaflet — your choice.
- **Places/store data:** your choice - see Section 5.

4. Functional Flow

Step 1 -> Portfolio Upload

- User uploads a CSV/XLSX ("PF file") of their own stores.

- Perform basic header validation before accepting the file (required columns present, correct types) and surface clear errors if it fails.
- Suggested columns: store_name, address, city, state, country, category, latitude (optional), longitude (optional).
- We'll provide a small sample PF file with the brief.

Step 2 -> Market Setup

On this screen the user configures the market before creating it:

- **Location** - select Country → State → City from dropdowns. These are **not** free text; seed the DB with a small fixed list of defaults, e.g.:
 - India / Karnataka / Bengaluru
 - India / Maharashtra / Mumbai
 - India / Delhi / New Delhi
- **Categories** — multi-select from a small seeded list of store categories (map these to real Google Places / OSM category types), e.g. Supermarket, Pharmacy, Hypermarket, Grocery Store, Convenience Store. Only stores matching the selected categories should be discovered.
- Behind the scenes, once a city is selected, generate a **rectangular boundary (bounding box)** for it using a free geocoding source (e.g. OpenStreetMap Nominatim, or your API of choice), and render it on the map. All store discovery in Step 3 must be constrained to this boundary, nothing outside it should be pulled in.
- The rectangle must be **draggable/resizable** on the map, the user can shrink or expand it before submitting, and the final boundary used for discovery is whatever the user leaves it at.
- **Cost guardrail**: a larger boundary means more places-API calls (and more cost) when Step 3 runs. Show the current boundary area live as the user resizes it, cap it at **30 sq km**, and disable "Create Market" until the area is at or under that limit.

Step 3 -> Market Creation

- On submit, call an API to create the market. This kicks off:
 1. Geocoding any portfolio rows missing lat/long and which are falling within the boundary.
 2. Discovering stores inside the boundary, filtered to the selected categories, via a places API.
 3. Persisting everything against the new market.

Step 4 -> Market Dashboard

For a given market, show:

- **Map view** with independent, toggleable layers/filters:
 - Discovered (scraped) stores
 - Portfolio stores inside the boundary

- Portfolio stores that fall **outside** the boundary (still uploaded, but geographically not part of this market)
- **List view** of stores (name + category at minimum), alongside the map.

5. Data Sources

Use whichever combination gets you a working, defensible system:

- **Places/store discovery:** Google Places API (New) is a good option — it has a free tier (~10,000 requests) plus \$300 in trial credit (~7,000 more). OpenStreetMap/Overpass is a fully free alternative. Any other places API is fine too.
- We'd prefer you integrate against a **real API** rather than fully stub it — that's where a lot of the interesting design decisions live (rate limits, pagination, error handling, cost awareness). If API access becomes a genuine blocker (e.g. billing setup issues), stubbing with realistic fixture data is an acceptable fallback — just call it out in your README and we'll evaluate the architecture around it the same way.
- **Boundary/geocoding:** OpenStreetMap Nominatim (free) or an equivalent free-tier geocoding API.

6. Scope

In scope (core):

- Steps 1–4 above, end to end.
- Basic auth is not required — assume a single user/session.

Out of scope (bonus, only if time remains):

- **Store matching** — for each portfolio store, check whether a discovered store exists within **150 meters**; if so, mark it "matched." Add a 4th map filter/layer for matched stores.
- Anything beyond a single market at a time in the UI (e.g. market comparison, editing an existing market's boundary).

Please don't spend core time on the bonus item at the expense of the core flow — a clean, well-architected core submission beats a rushed one that also attempts matching.

7. Deliverables

- A git repo (zip or link) with backend + frontend.
- README.md with: setup/run instructions, a short note on your architecture decisions, and any known limitations or shortcuts taken due to time.
- Sample PF file provided by us (attach separately).
- Tests, if included, runnable via a single documented command.